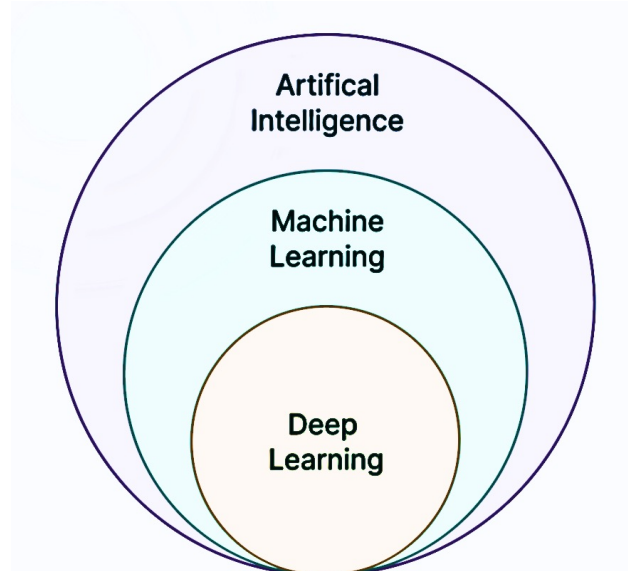


Module 4a

Architecture, Design and Programming of Artificial Neural Network I

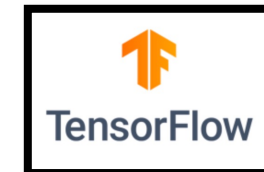


Prof. Emmanuel ADETIBA

**Electrical & Information Engineering Department, College of Engineering,
Covenant University, Ota, Nigeria.**



PROF E. ADETIBA - ARTIFICIAL INTELLIGENCE AND
APPLICATIONS



4.1 Artificial Neural Network(ANN) Definition

An ANN is an information processing model that was inspired from the understanding of biological nervous system. ANN was developed as a generalization of mathematical models of neural biology based on the following assumptions:

- a) Neurons are simple units in a nervous system at which information processing occurs.
- b) Incoming information are signals that are passed between neurons through connection links.

- c) Each connection link has a corresponding weight which multiplies the transmitted signal.
- d) Each neuron applies an activation function to its net input which is the sum of weighted input signals to determine the output signal.

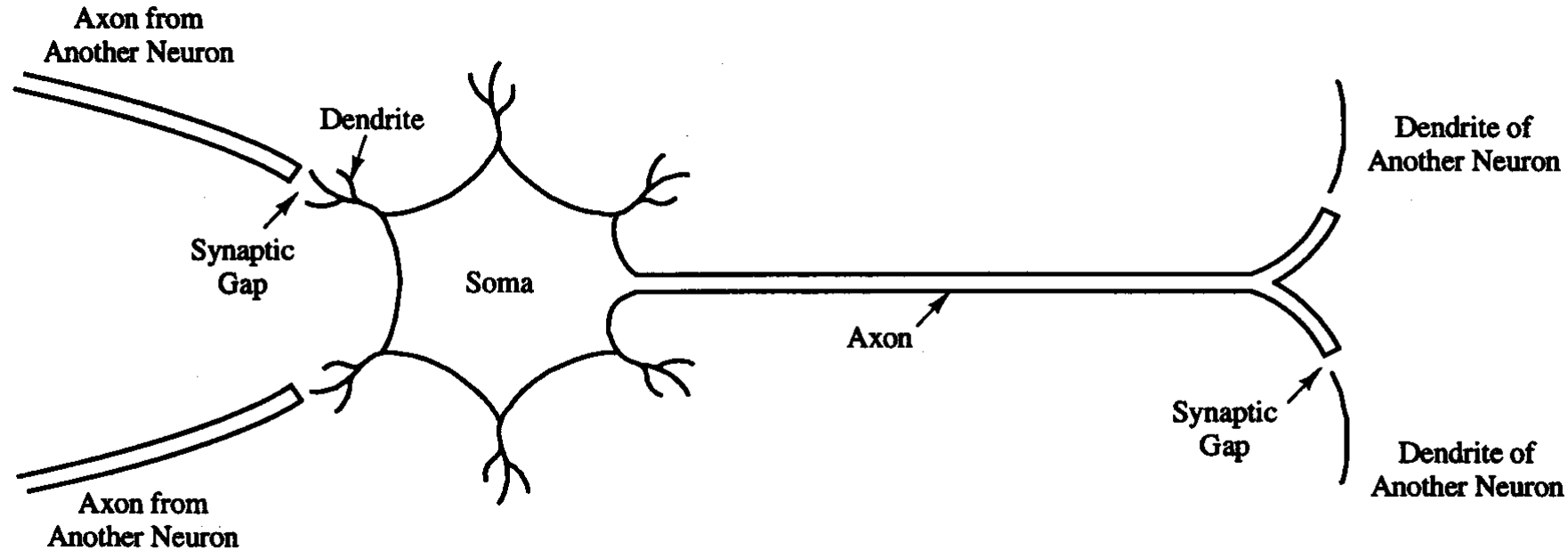


Figure 4.1: Biological Neuron

(Source: Fundamental of Neural Networks: Architectures, Algorithms and Application by Laurene Fausett,)

4.1.1 Artificial Neural Network (ANN) Characteristics

❖ Some ANN properties that are inspired by biological neural network are:

a) Non-linearity: Interconnection of nonlinear neurons and the non-linearity is distributed throughout.

ii. Input-Output mapping: Ability to learn with or without a Teacher.

iii. Adaptability: Can adapt the free parameters (synaptic connections) to changes in the surrounding environment.

iv. Evidential response: Able to make decision with a measure of confidence.

v. Fault Tolerance: It experiences graceful degradation which implies that if a fault is small the degree of degradation will be small.

vi. VLSI implementability: The neurons in ANN are parallel computation units and can be implemented with very large scale integrated circuit (like ASIC, DSP, FPGA, TPU) and parallel computing platforms (such as Multi GPU, Multi-core CPU)

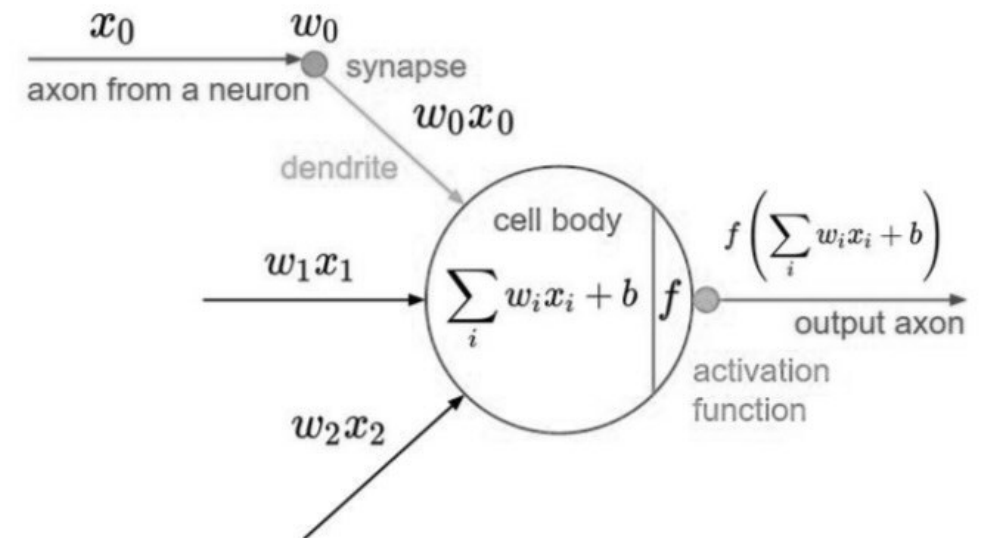
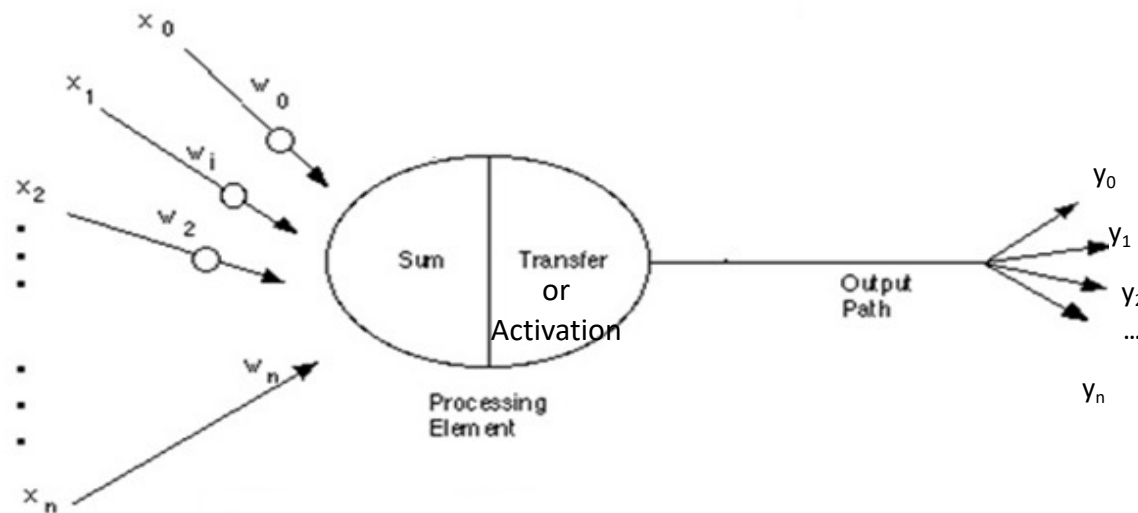
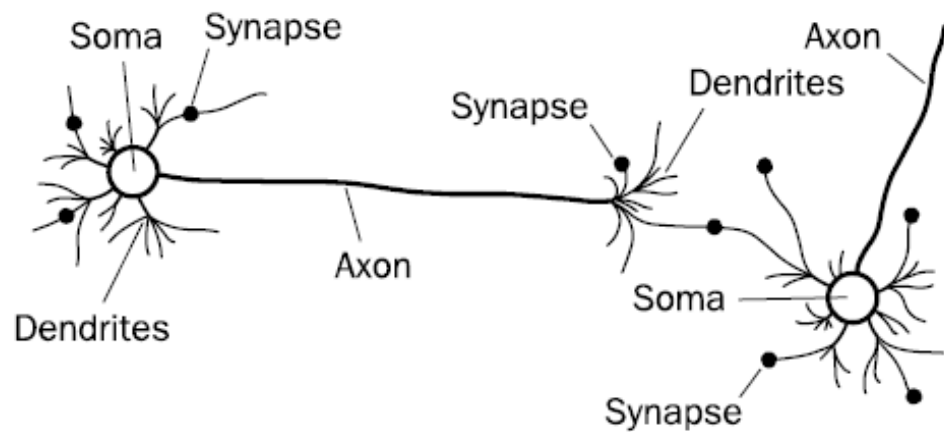
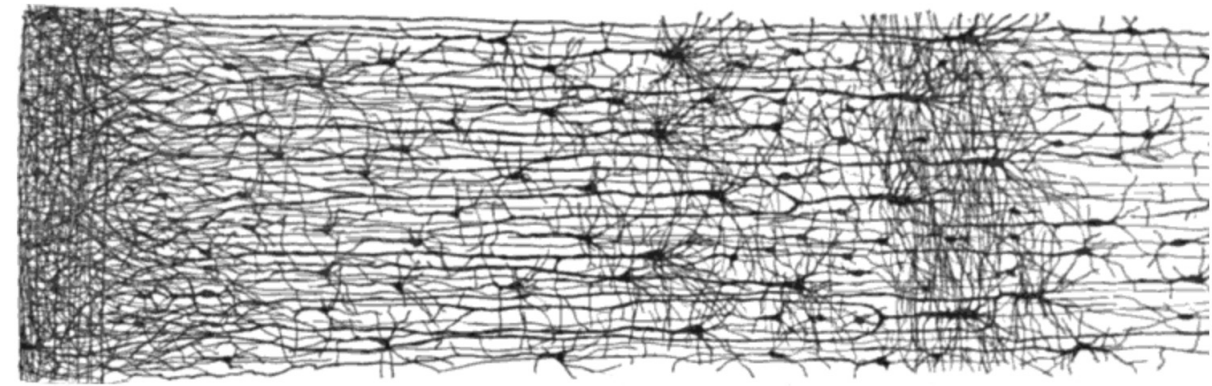


Figure 4.2: Artificial Neuron

PROF E. ADETIBA - ARTIFICIAL INTELLIGENCE AND
APPLICATIONS

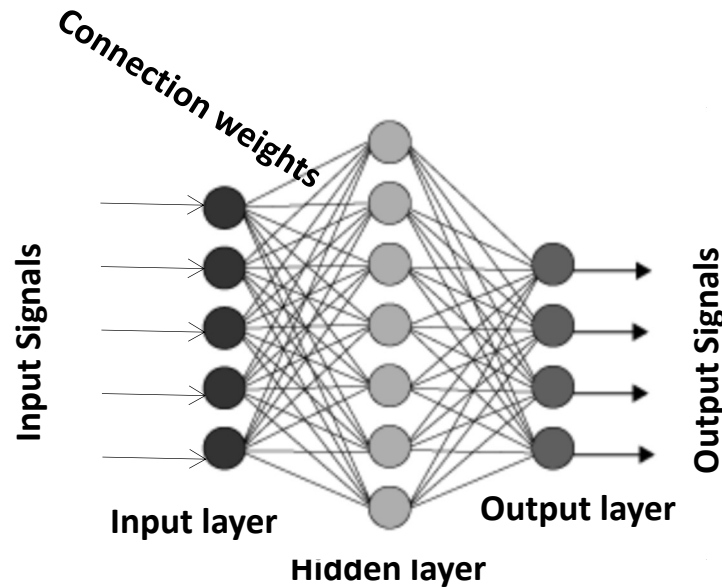


A) Simple Biological Neural Network

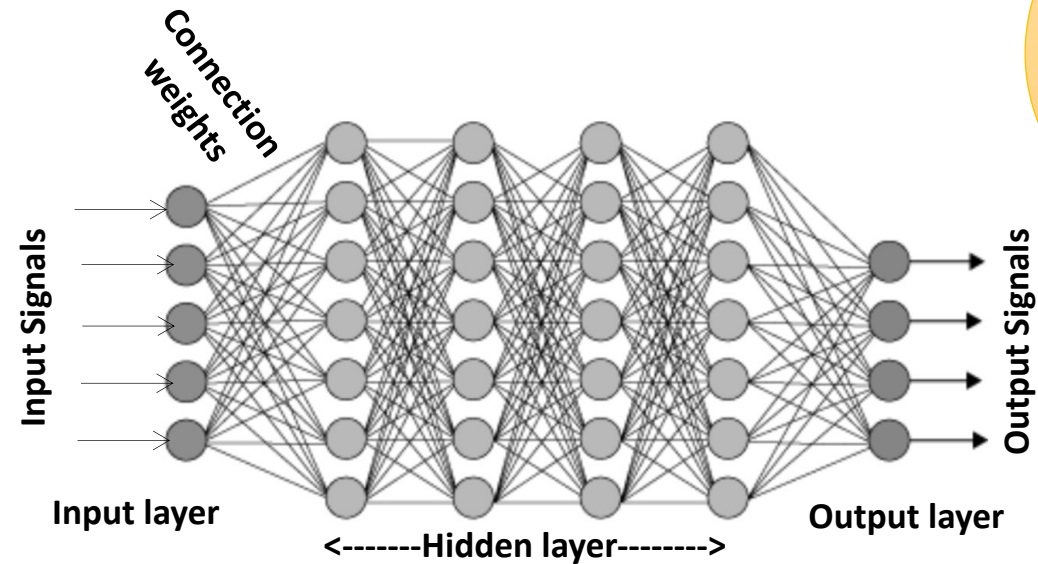


B) Multiple Layers in Biological Neural Network
(Source: Neural networks and deep learning by Aurélien Géron)

Figure 4.3: Biological Neural Network



A) Shallow Architecture



B) Deep Architecture

Between 14 to 16 billion neurons are found in the cerebral cortex.

Figure 4.4: Typical Artificial Neural Network (ANN) Architectures

4.1.2 Similarity/Analogy of Biological Neural Network with ANN

Table 4.1: Analogy Between Biological and Artificial Neural Networks

S/N	Biological Neural Network	Artificial Neural Network (ANN)
1	Soma	Neuron
2	Dendrite	Input
3	Axon	Output
4	Synapse	Weight

4.1.3 Differences Between the Biological and ANN

- i. **Parameters:** Human brains have tens of million times synapses than ANN connections.
- ii. **Operation Mode:** The human brain works **asynchronously** while ANN works **synchronously**.
- iv. **Learning Algorithm:** ANN uses gradient descent learning and its variants(as well as other learning algorithms) while how the human brain learns is not yet known.
- v. **Power consumption:** Human brain uses very little power compared to ANN.
- vi. **Learning Phases:** Human brain never stops learning as long as it is living, while ANN first trains then tests.

4.2 Activation Functions in Neurons

- ❖ The basic operation of an artificial neuron involves summing its weighted input signals and applying an activation (or transfer) function to generate an output.
- ❖ Many activation functions have been tested for artificial neurons but only a few have found practical applications.
- ❖ Traditional and modern activation functions that are commonly used in practical applications are described as follows:

1) *Identity function*

- This function provides an output which is equal to the neuron weighted input. It is also called ***linear function***. Neurons with this function are used for linear approximation. It is represented mathematically as:

$$f(z) = z \quad (4.1)$$

where z = sum of weighted inputs into the neuron. It is graphically represented in Fig 4.5.

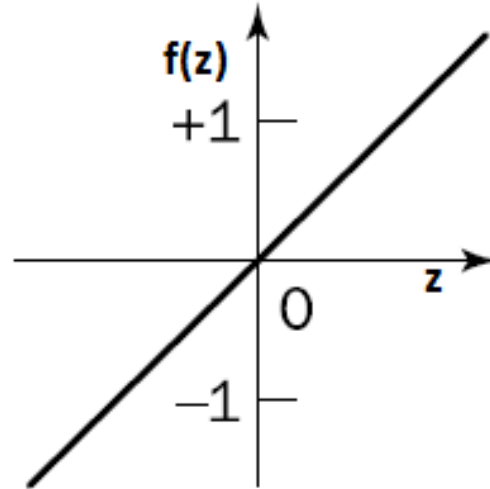


Fig 4.5: Identity function

2) Step functions

- They are also called the ***hard limit function***. They are often used in decision making neurons for classification and pattern recognition tasks. There are two types of step functions:

(a) Binary step function

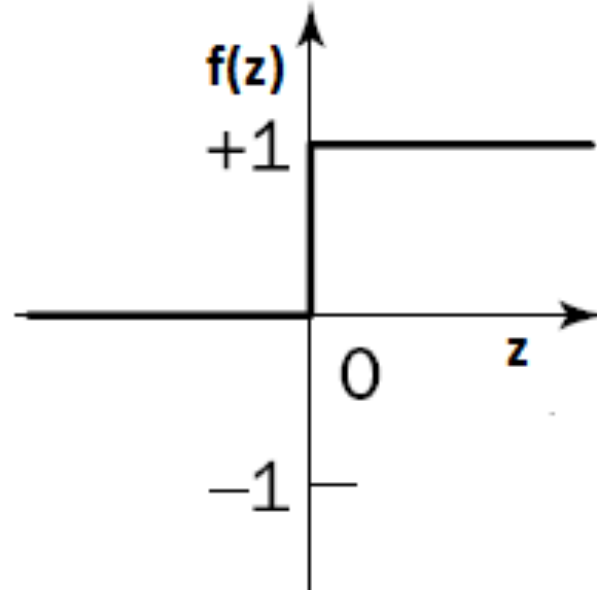
- This is also known as ***Threshold or Heaviside function***. It is represented mathematically as:

$$f(z) = \begin{cases} 1 & \text{if } z \geq T \\ 0 & \text{if } z < T \end{cases} \quad (4.2)$$

where z = sum of weighted inputs into the neuron

T = threshold

❖ The function is represented graphically in Fig 4.6



PROF E. ADETIBA - ARTIFICIAL INTELLIGENCE AND APPLICATIONS
Fig. 4.6: Binary step function

(b) Bipolar step function

- This is also called ***sign function***. It is represented mathematically as:

$$f(z) = \begin{cases} 1 & \text{if } z \geq T \\ -1 & \text{if } z < T \end{cases} \quad (4.3)$$

and the graph of the function is shown in Fig. 4.7

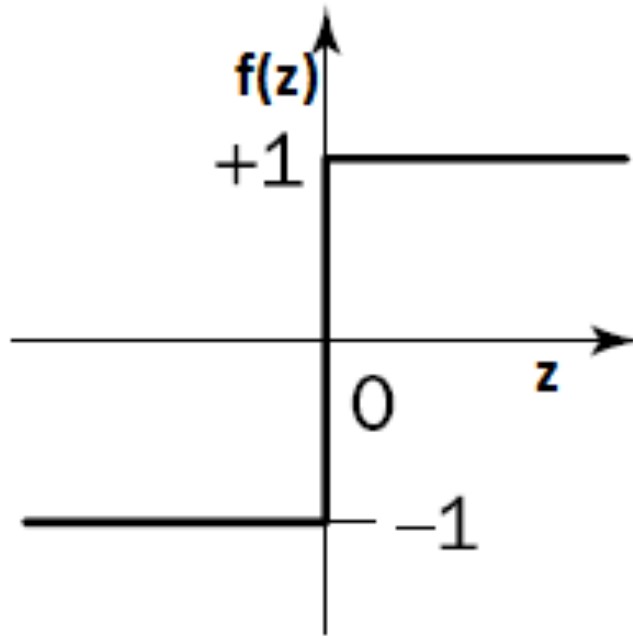


Fig. 4.7: Bipolar step function

(3) Sigmoid functions

- These are S-shaped curves that are very useful in multi-layer networks. Some of the available types are:

(a) Binary sigmoid function

- This is also called ***logistic function***. It is a sigmoid function with range from 0 to 1 and used for neural networks in which the desired output are either binary or are in the interval between 0 and 1. It is represented as:

$$f(z) = \frac{1}{1 + \exp(-\sigma z)} \quad (4.4)$$

where z = sum of weighted inputs

σ = steepness parameter

❖ The graph of binary sigmoid function is shown in Fig 4.8.

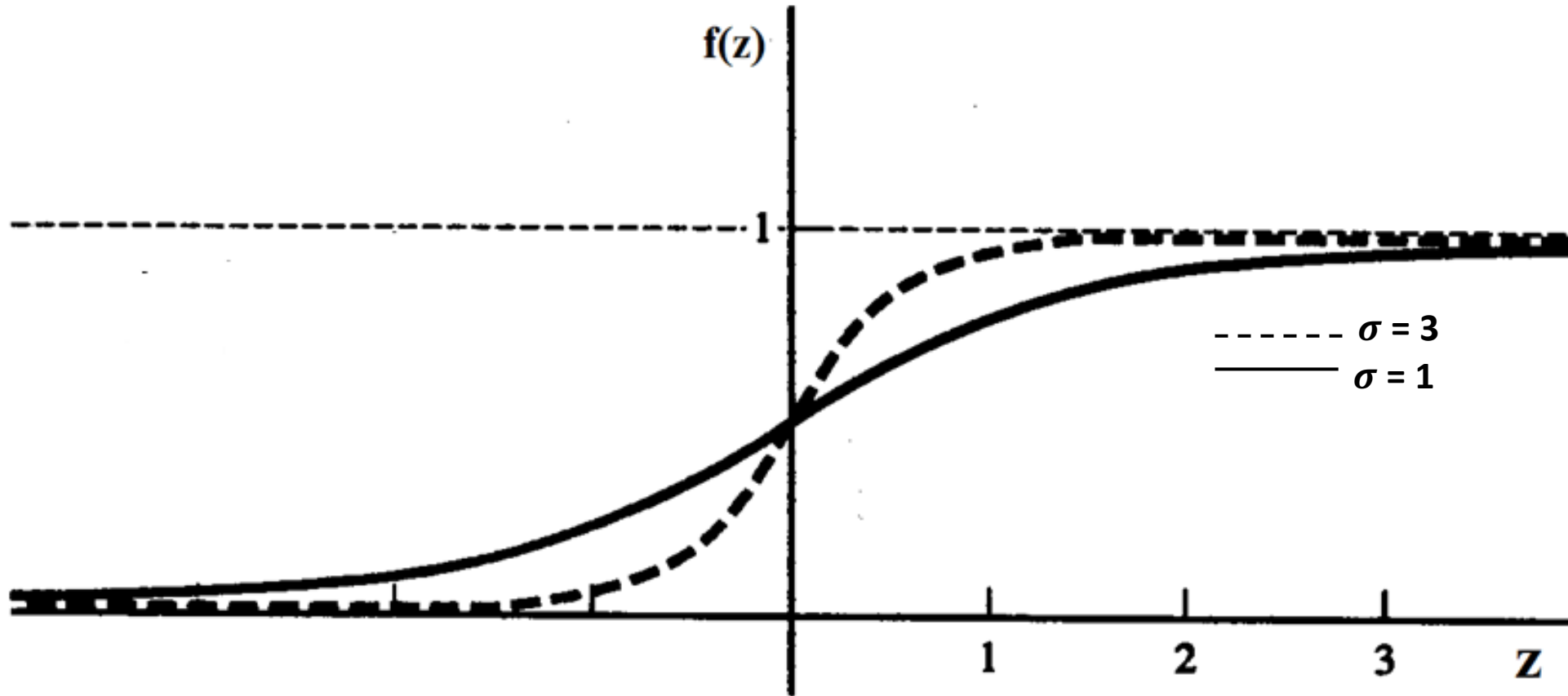


Fig. 4.8: Binary sigmoid function

(b) Bipolar sigmoid function

- The range of output for this function is from -1 to 1. It is closely related to the ***hyperbolic tangent function***. It can be used as the activation function when the desired output range is between -1 and 1. It is generically represented as:

$$f(z) = \frac{1 - \exp(-\sigma z)}{1 + \exp(-\sigma z)} \quad (4.5)$$

❖ For $\sigma = 1$, bipolar sigmoid function is:

$$f(z) = \frac{1 - \exp(-z)}{1 + \exp(-z)} \quad (4.6)$$

❖ The graph of bipolar sigmoid function is shown in Fig. 4.9.

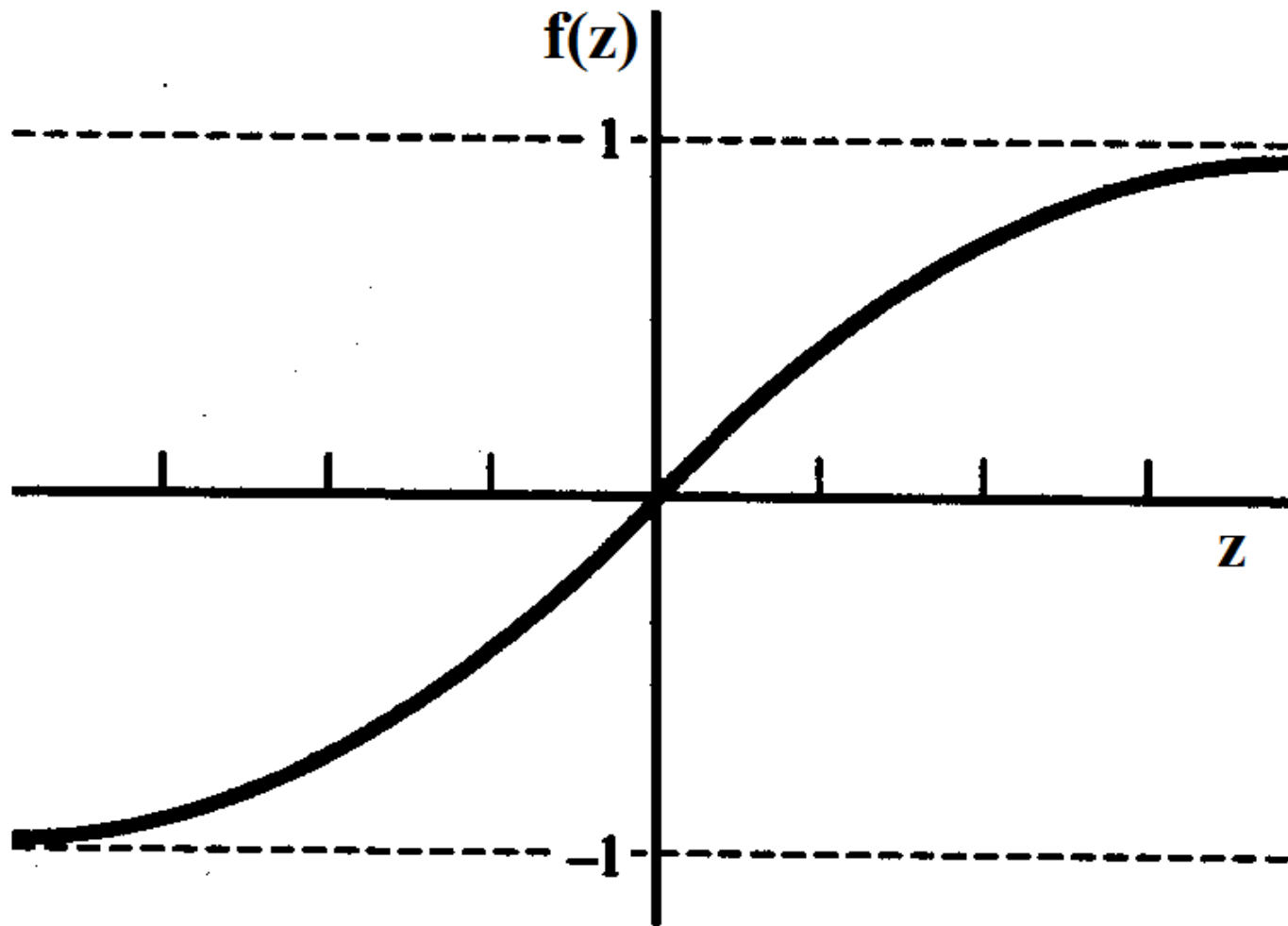


Fig. 4.9: Bipolar sigmoid function

c) Softmax function

- Softmax function is often described as a combination of multiple sigmoids.
- The function can be used for multiclass classification problems/multiclass models, where it returns the probabilities of each class, with the target class having the highest probability.
- The output of softmax function is a range of values between 0 and 1, with the sum of the probabilities equal to 1. It is represented mathematically as:

$$f(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K \quad (4.7)$$

- For example, if you have three classes, there would be three neurons in the output layer.

Example 4.1

Assuming you got the output from three neurons as [1.2, 0.9, 0.75], apply the **softmax** function over these values. Show that the outputs of the Softmax operation (probability) for the 3 classes sums up to 1.

Solution

$$f(z_1) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(1.2)}{\exp(1.2) + \exp(0.9) + \exp(0.75)} = 0.42$$

$$f(z_2) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(0.9)}{\exp(1.2) + \exp(0.9) + \exp(0.75)} = 0.31$$

$$f(z_3) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(0.75)}{\exp(1.2) + \exp(0.9) + \exp(0.75)} = 0.27$$

- The graph of **softmax** activation function is shown in Fig. 4.10

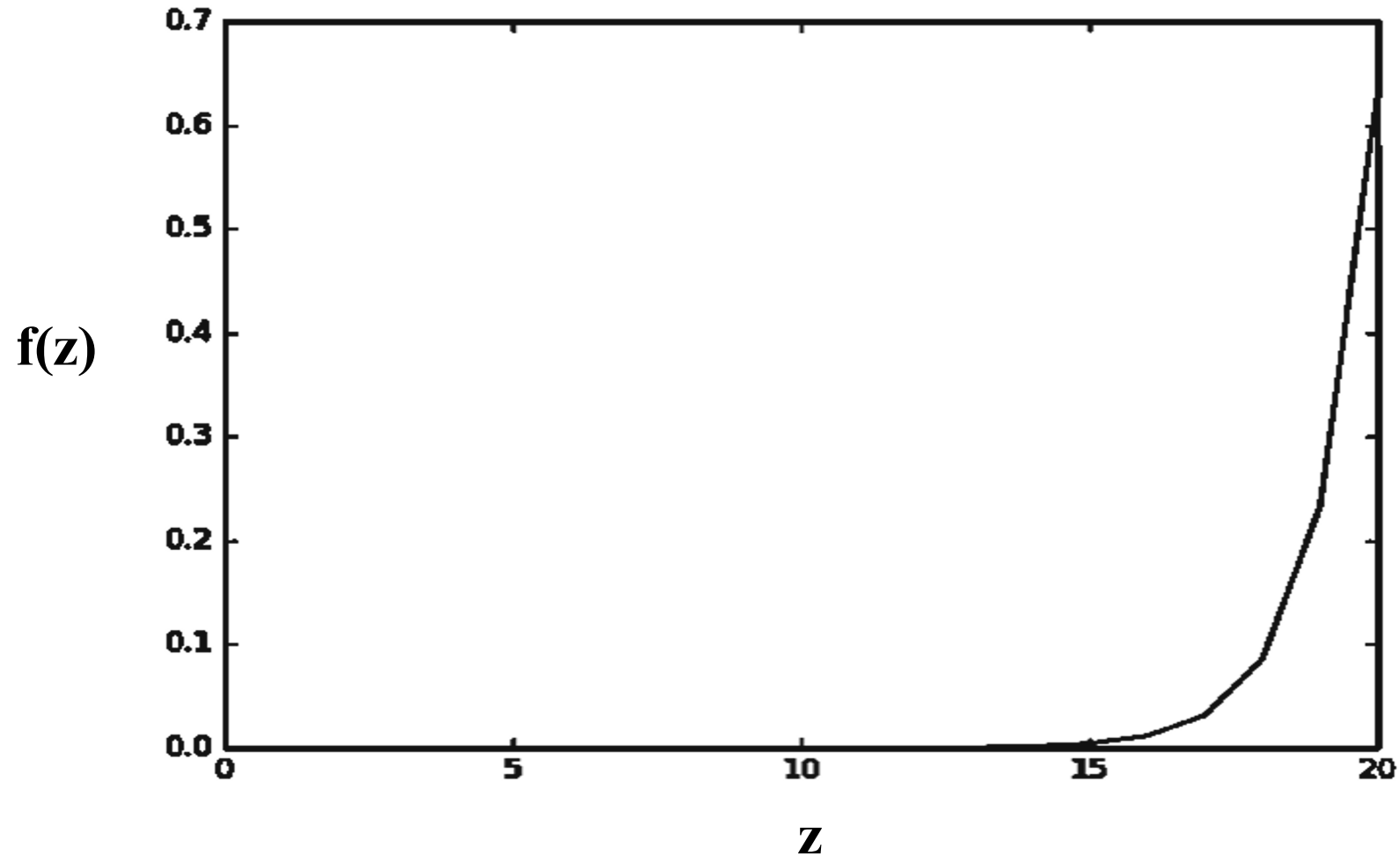


Fig. 4.10: Softmax function

(3) Rectified Linear Unit functions

- When neural networks are large(for example Deep Neural Networks), both binary and bipolar sigmoid functions suffer from the *vanishing gradients* problem, which implies that the gradients computed during backpropagation are smaller than 1.
- The effect of this problem is that most upstream layers in the network learn very slowly or stops learning, which severely impacts learning deep ANN architectures. Rectified linear unit function helps to solve this problem. Variants of rectified linear unit functions are:

a) Standard ReLU function

- The standard Rectified Linear Unit (ReLU) activation function was proposed by Nair and Hinton in 2010 to address the *vanishing gradients* problem.
- Since its development, it has been the most widely used activation function for deep learning applications with state-of-the-art performance.

- The operation of ReLU is closer to the way our biological neurons work by performing a threshold operation to each input element.
- It is represented mathematically as:

$$f(z) = \max(0, z) = \begin{cases} z_i & \text{if } z_i \geq 0 \\ 0 & \text{if } z_i < 0 \end{cases} \quad (4.8)$$

- This function rectifies the values of the inputs that are less than zero by forcing them to zero (thus eliminating the vanishing gradient problem) and if the input is greater than 0, the output is equal to the input.

- Other striking advantages of ReLU are: *i) scale-invariance, ii) sparse activation, iii) better gradient propagation, iv) biological plausibility and v) efficient computation* (i.e. only comparison, addition and multiplication are involved).
- The major drawbacks of ReLU function are: i) it is non differentiable at Zero and ii) it suffers from the **dead ReLU** problem, in which some ReLU neurons die for all inputs and remain inactive regardless of the input.
- The graph of **ReLU** activation function is shown in Fig. 4.11.

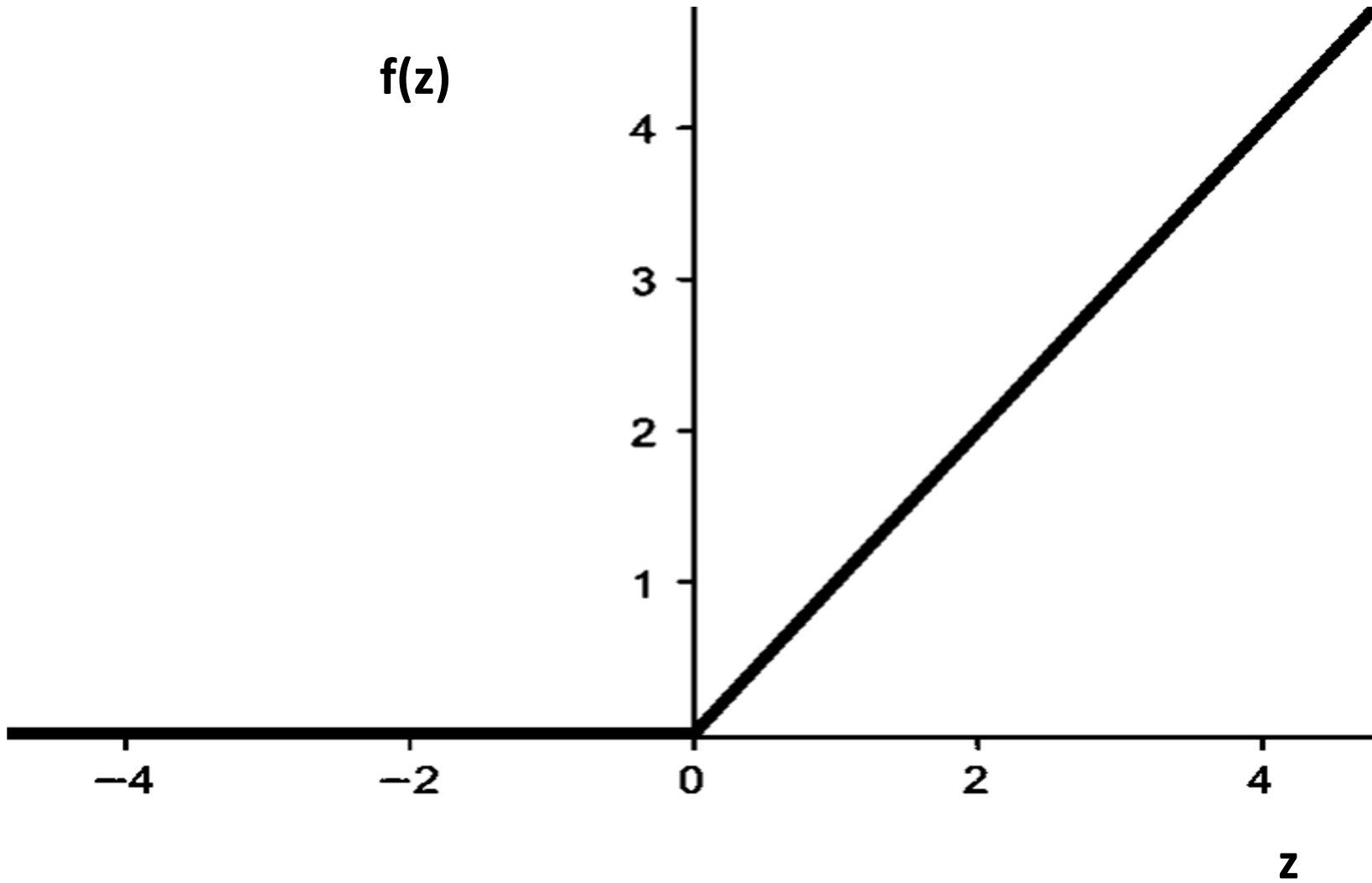


Fig. 4.11: ReLU function

b) Leaky ReLU function

- The Leaky ReLU (LReLU) is an activation function that minimizes neural networks' sensitivity to the **dead ReLU** problem such that the gradients will not be zero at any time during training.
- LReLU computes the gradient with a very small constant value for the negative gradient α in the range of 0.01.
- The mathematical representation of LReLU is:

$$f(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases} \quad (4.9)$$

- The LReLU has an identical result when compared to the standard ReLU with an additional benefit that it has non-zero gradients over the entire duration.
- The graph of **LReLU** activation function is shown in Fig. 4.12.

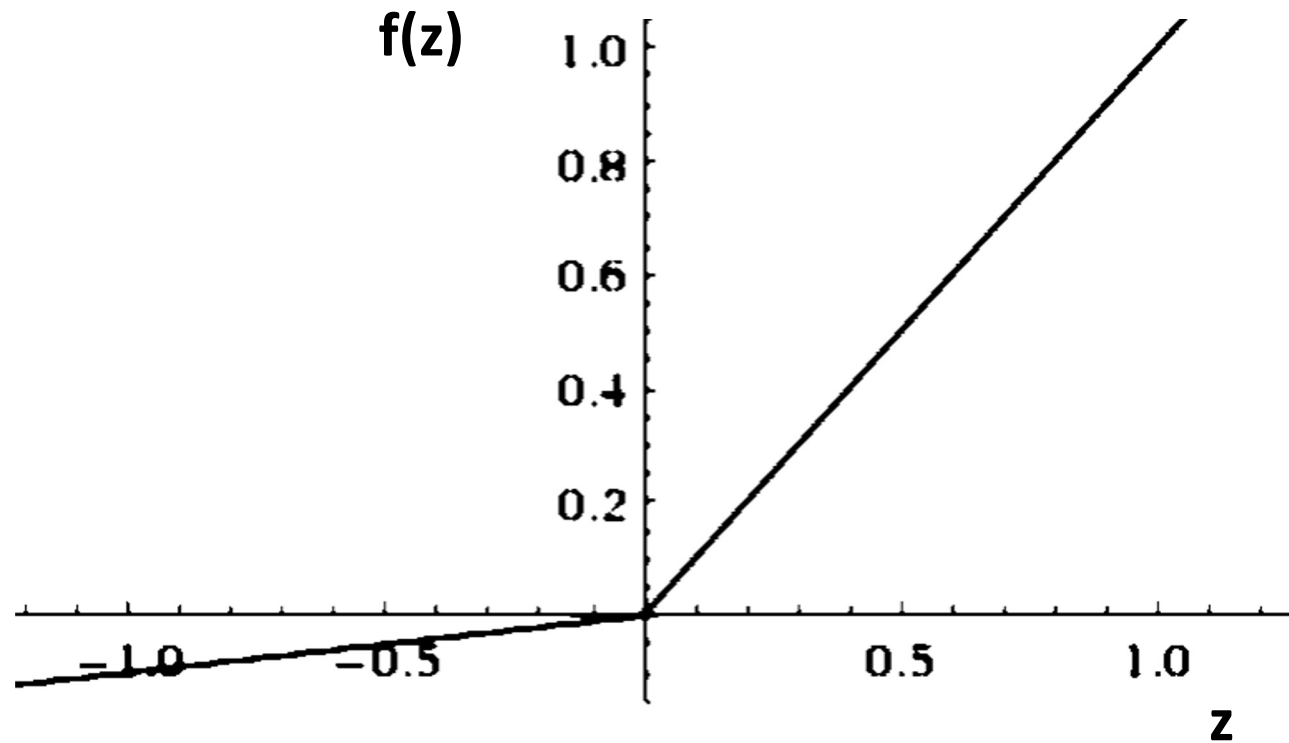


Fig. 4.12: LReLU function

c) Other ReLU functions

- Parametric ReLU (PReLU), Randomized leaky ReLU (RReLU) and S-shaped ReLU are variants of ReLU that have been developed and reported to have performance improvement over LReLU in addressing the dead ReLU problem.

- Table 4.2 shows the activation functions used by state-of-the-art deep neural network architectures.

Table 4.2: Activation functions used by state-of-the-art networks

NN Models	Hidden Layers	Output Layer	Reference
AlexNet	ReLU	Softmax	Krizhevsky et al.,2012
NiN	No activation	Softmax	Lin et al., 2013
ZFNet	ReLU	Softmax	Zeiler & Fergus,2013
VGGNet	ReLU	Softmax	Simonyan & Zisserman,2015
SegNet	ReLU	Softmax	Badrinarayanan et al., 2015
GoogleNet	ReLU	Softmax	Szegedy et al.,2015
SqueezeNet	ReLU	Softmax	Golkov et al., 2016
ResNet	ReLU	Softmax	He et al., 2016
ResNetXt	ReLU	Softmax	Xie et al., 2017
MobileNets	ReLU	Softmax	Howard et al., 2017
SeNet	ReLU	Sigmoid	Fu et al., 2017

4.3 McCulloch-Pitt Neuron Model

- ❖ The McCulloch-Pitt (MCP) neuron is the first formal definition of a synthetic neuron model based on the highly simplified consideration of the biological neuron.
- ❖ It was developed by Warren McCulloch and Walter Pitts in 1943.
- ❖ The model had a huge influence on the thought and studies that led to modern digital computer because it outlined the first formal model of an elementary computing neuron.

❖ The characteristics of MCP neuron model are as follows:

- (i) It allows binary 0 or 1 states only.
- (ii) The neurons can be connected by directed weighted path.
- (iii) The connected path can be excitatory or inhibitory.
- (iv) Excitatory connections have positive weight and inhibitory connections have negative weights.
- (v) There will be same weights for the excitatory connections entering into a particular neuron.

(vi) The neuron is associated with the threshold value and the neuron fires if the net input to the neuron is greater than the threshold

(vii) It takes only one time step for a signal to pass over one connection link.

❖ The MCP is a Threshold neuron(i.e. it uses threshold activation function) and hence, the mathematical model is the same as Equation (4.10) i.e. :

❖ The architecture of MCP is shown in Fig. 4.13.

$$f(z) = \begin{cases} 1 & \text{if } z \geq T \\ 0 & \text{if } z < T \end{cases} \quad (4.10)$$

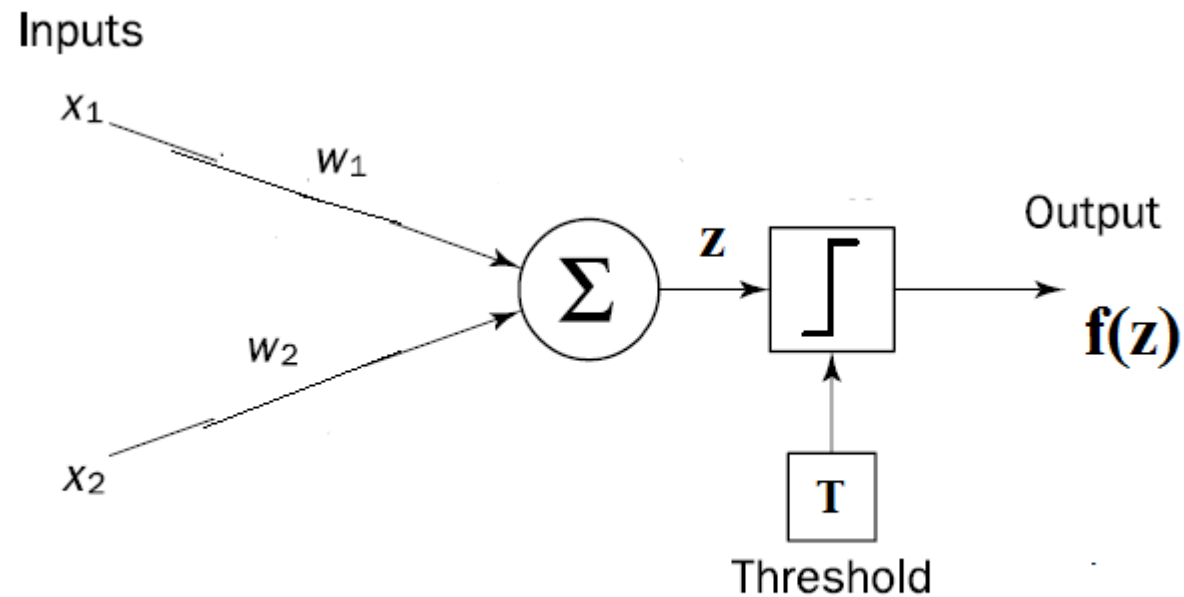


Fig 4.13: Architecture of McCulloch-Pitt Neuron

- ❖ Simple logical functions can be implemented directly with a single MCP neuron. Weights and thresholds can be set to yield neurons which realise the logical functions AND, OR and NOT.
- ❖ The output value 1 can be associated with the logical value *true* and 0 with the logical value *false*.

(a) “AND” logical function

Table 4.3: “AND” Truth Table

x_1	x_2	$x_1 \text{ AND } x_2$
0	0	0
0	1	0
1	0	0
1	1	1

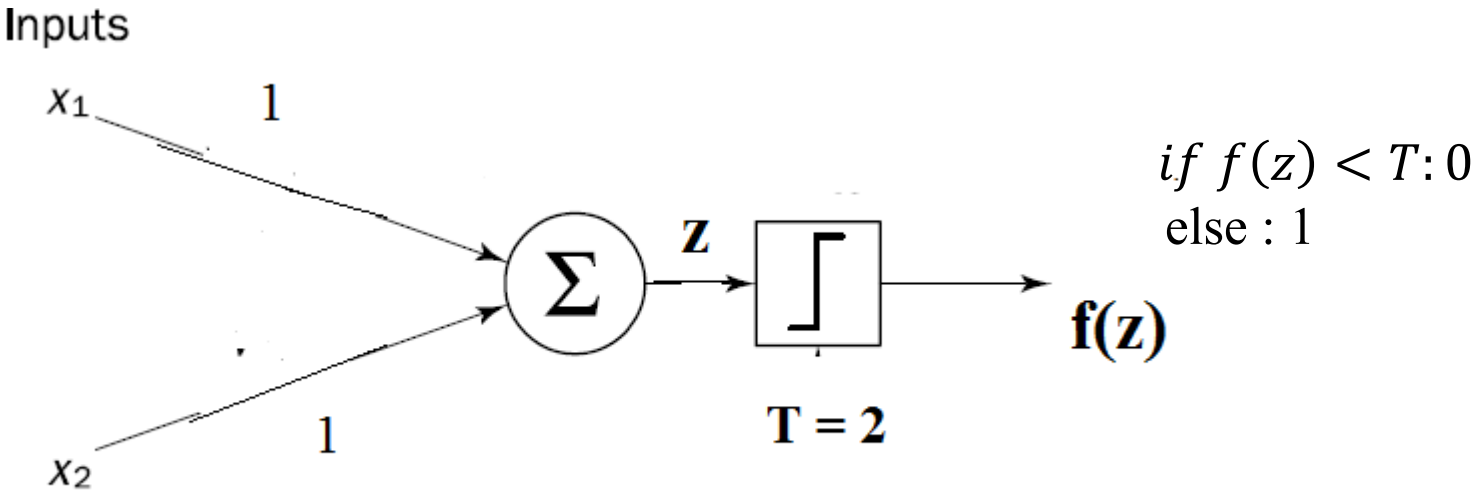


Fig 4.14: Architecture of McCulloch-Pitt Neuron for “AND” Logical Function

(a) “OR” logical function

Table 4.4: “OR” Truth Table

x_1	x_2	x_1 OR x_2
0	0	0
0	1	1
1	0	1
1	1	1

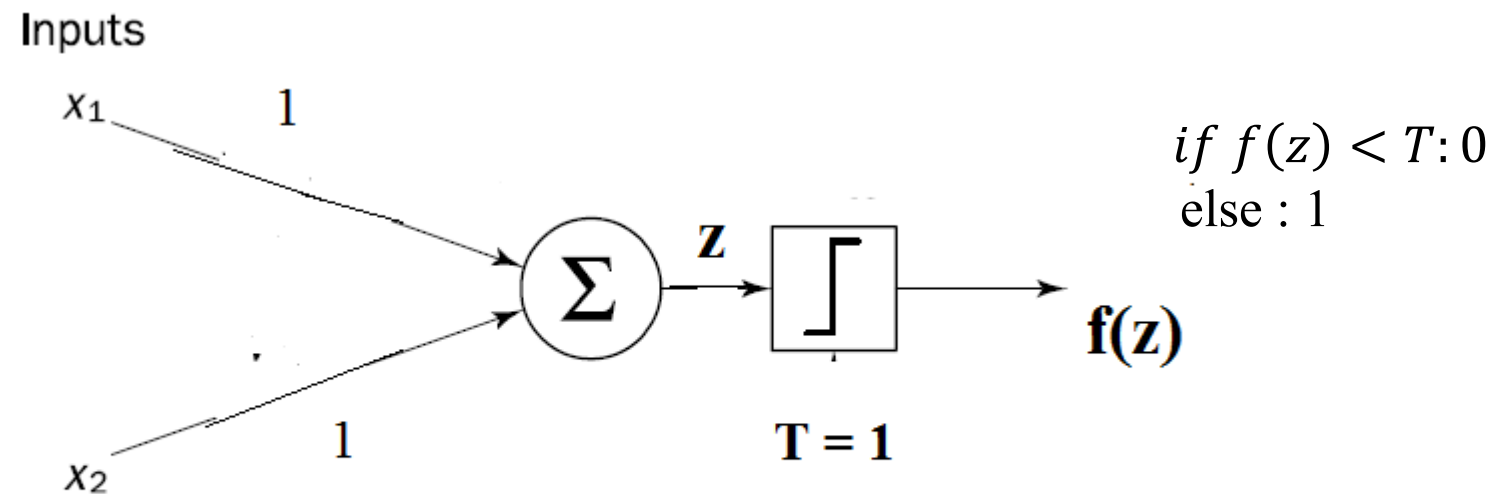


Fig 4.15: Architecture of McCulloch-Pitt Neuron for “OR” Logical Function

- The **AND** and **OR** logical operations can be geometrically represented as shown in Fig.4.16.
- A single MCP neuron splits the input points into two halves.
- All inputs which produce an output 0 will be on one side of the line and all inputs which produce an output 1 will lie on the other side of this line.

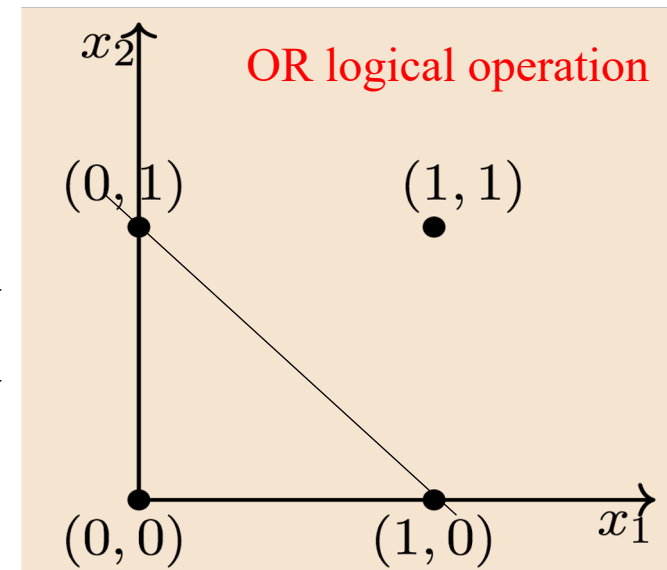
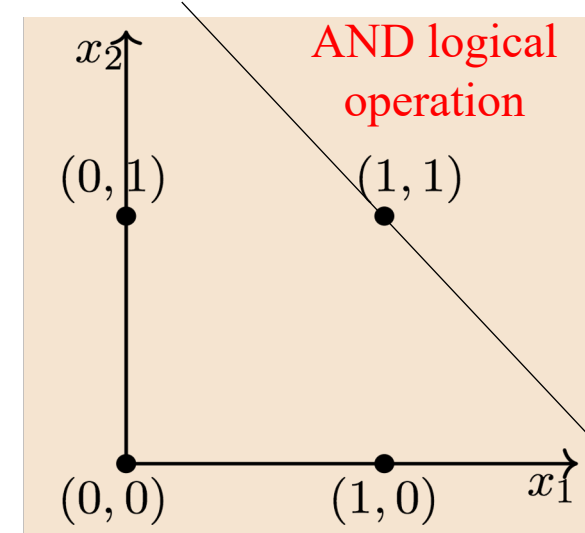


Fig. 4.16: Geometrical Representations of Logical AND and OR

- ❖ As shown in the foregoing, a single MCP neuron can be used to represent boolean functions which are linearly separable.
- ❖ A boolean function is said to be **Linearly Separable** if there exists a line (or plane) such that all inputs which produce a 1 lie on one side of the line (or plane) and all inputs which produce a 0 lie on other side of the line (or plane).

Module 4a – Assignment 1

Assuming the weighted sum of five neurons is given as:

[1.2, 0.9, 0.75, 0.8, 1.5]

use Python programming language with relevant framework to compute the output of the neurons using the following activation functions:

- i) Softmax
- ii) ReLU
- iii) Bipolar sigmoid with $\sigma = 1$

NOTE:

Your notebook **MUST** be submitted on MOODLE on or before the next class. Your notebook should be named as *fullname_matricnumber_mod4a.ipynb*

4.5 The Perceptron: Architecture and Implementation

- ❖ The Perceptron was the first supervised neural model developed by Frank Rosenblatt in 1958.
- ❖ This model created a great interest during the 60s due to its capability to learn and recognize simple patterns.
- ❖ The operation of Rosenblatt's perceptron is based on the *linear threshold unit* (LTU) neuron model, which consists of a linear combiner followed by a hard limiter.
- ❖ The weighted sum of the inputs is applied to the hard limiter (or Heaviside step function), which produces an output equal to +1 if the input is positive and 0 if it is negative.

$$f(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \quad (1)$$

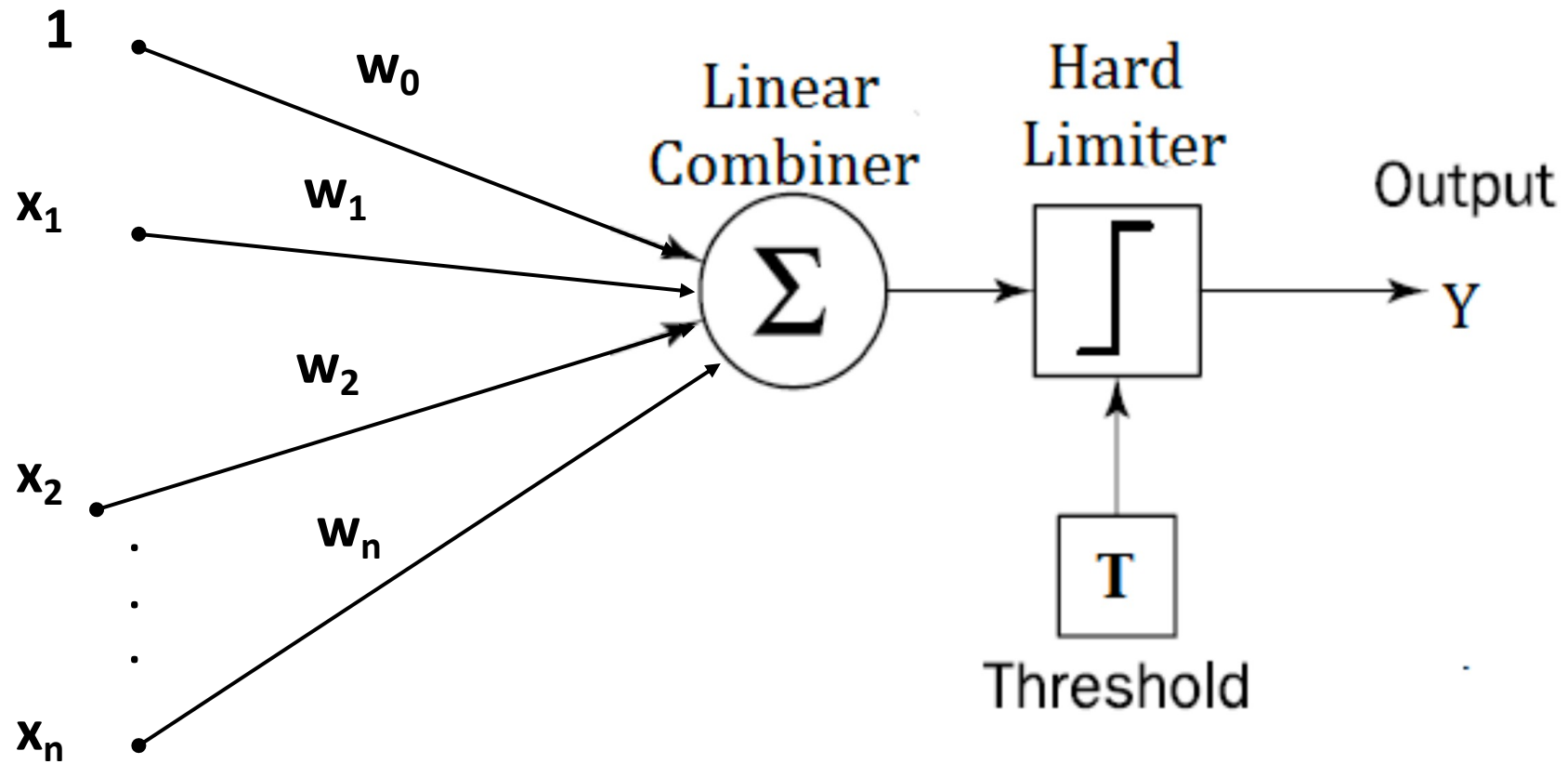


Fig. 4.17: The Frank Rosenblatt Perceptron

- ❖ The goal of the perceptron is to classify inputs $x_1, x_2, \dots, x_n$ into one of two (binary) classes.
- ❖ For elementary perceptrons, the n -dimensional space is divided by a hyperplane into two decision regions.
- ❖ For the Perceptron to learn the classification task, the weights are adjusted based on the Perceptron rule:

$$w_i(p+1) = w_i(p) + \alpha x_i(p) e(p) \quad (2)$$

$$\text{but } e(p) = y_d(p) - y(p) \quad (3)$$

$\therefore$

$$w_i(p+1) = w_i(p) + \alpha x_i(p) (y_d(p) - y(p)) \quad (4)$$

❖ Equation (3) is the Perceptron learning rule.

where $p = 1, 2, 3, \dots$

α is the learning rate, which is usually set to be less than unity.

❖ From this rule, the Perceptron training algorithm for classification task is developed. During training, the parameters (i.e. weights and biases) of the perceptron are adjusted to minimize the error (e) between the predicted output (y_d) and the true output (y) for a given set of training examples ($x_1, x_2, \dots, x_n$).

4.4.1 The Perceptron Training Algorithm

Step1: Initialization

- Set initial weights $w_1, w_2, \dots, w_n$ and threshold T to random numbers in the range $[-0.5, 0.5]$.

Step2: Activation

- Activate the Perceptron by applying inputs $x_1, x_2, \dots, x_n$ and desired output y_d .
- Calculate the actual output at iteration $p = 1$

$$y(p) = \text{step} \left[\sum_i^n x_i(p) w_i(p) - T \right]$$

- Where n is the number of the Perceptron inputs and *step* is a step activation function(i.e. hardlimit).

Step3: Weight Training

- Update the weights of the Perceptron

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

where $\Delta \mathbf{w}_i(\mathbf{p})$ is the weight correction at iteration p and

$$\Delta w_i(p) = \alpha x_i(p) e(p)$$

Step4: Iteration

- Increase iteration p by one and go back to Step2.
- Repeat the process until convergence.

❖ The primary limitations of the Perceptron model are:

- i) It can only output a binary number(0 or 1) due to the threshold (hardlimiter) activation function that it uses.
- ii) It can only be used to solve linearly separable problems (e.g. AND, OR, NOT logics).

❖ Despite the limitations of the Perceptron model, it has paved the way for more complex neural networks used in various deep learning architectures and foundational ML models today.

Example 4.2

Develop a Python Code with Sklearn framework for classification of the first two species of the Iris Dataset.

Solution

```
#Import required libraries
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix

#Load the Iris dataset
iris = load_iris()

#View the iris data keys
iris.keys()
```

```
#it is a key of dictionary iris
iris.target_names

##Convert data and target into a data frame.

#Extract the First 100 Features
X = pd.DataFrame(data = iris.data[:100,:], columns = iris.feature_names)
X.head(100) #Inspect the features

#Target
y = pd.DataFrame(data=iris.target[:100], columns = ['irisType'])
y.head(100)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Confusion Matrix:\n",confusion_matrix(y_test, y_pred))
```

Module 4a – Assignment 2

Use TensorFlow framework to implement Example 4.2

NOTE:

Your notebook **MUST** be submitted on MOODLE on or before the next class.
Your notebook should be named as *fullname_matricnumber_mod4a.ipynb*